

Comunicación de WALLI versión 1

Juan Sebastián Caliman García
Código: 20181005127
Andrés Felipe Cepeda Carvajal
Código: 20191005056
Carlos Andrés Rey Velásquez
Código: 20202005150
Karen Tatiana León Olaya
Código: 20202005152
Luis Ángel Camargo Guzmán
Código: 20221005086
Nicolás Alejandro Romero Montoya
Código: 20201005035
Santiago Murillo Soto
Código: 20201005185

Facultad de Ingeniería, Universidad Distrital Francisco José de Caldas
Bogotá D.C., Colombia

Abstract—Este documento presenta la primera versión del sistema de comunicación entre un computador central y un conjunto de 8 carros recolectores tipo WALLI. El sistema permite el envío de comandos en formato JSON para ejecutar coreografías sincronizadas entre los carros, incluyendo movimientos, control de brazo robótico y procesamiento de imagen.

Index Terms—WALLI, comunicación JSON, red WiFi, UART, coreografía robótica, imagen RGB565, Flask, MicroPython

I. INTRODUCCIÓN

Los sistemas de vehículos cooperativos representan un reto en la ingeniería moderna, especialmente cuando se requiere sincronizar acciones entre múltiples agentes. En esta primera fase se implementa una arquitectura que permite enviar instrucciones desde un computador central a ocho carros WALLI de forma simultánea, mediante paquetes UDP y estructuras JSON. Cada carro cuenta con un brazo articulado, capacidad de desplazamiento y una cámara integrada para captura de imagen.

II. OBJETIVO GENERAL

Desarrollar un sistema de comunicación para controlar de forma sincronizada ocho unidades WALLI desde un computador central, utilizando redes WiFi y comandos estructurados en JSON.

III. OBJETIVOS ESPECÍFICOS

- Implementar un punto de acceso central que reciba y distribuya paquetes UDP hacia los carros.
- Diseñar el protocolo de comunicación JSON para instrucciones de movimiento y brazo robótico.
- Permitir la recepción y procesamiento de imágenes capturadas por las cámaras de los carros.

- Validar la sincronización de movimientos mediante una coreografía programada.

IV. FUNDAMENTO TEÓRICO

A. Raspberry Pi Pico W en Robótica

El Raspberry Pi Pico W ofrece características clave para este proyecto:

- Microcontrolador RP2040 dual-core a 133MHz
- 264KB de SRAM y 2MB de memoria flash
- Conectividad WiFi 802.11n mediante chip Infineon CYW43439
- Soporte para MicroPython con librerías de red
- Múltiples interfaces (UART, I2C, SPI, PWM)
- Bajo consumo energético (ideal para robots móviles)

B. Comunicación WiFi en Pico W

La comunicación implementada utiliza:

- **UDP/IP:** Para envío de comandos (baja latencia)
- **HTTP:** Para transferencia de imágenes (fiabilidad)
- **Manejo de sockets:** Implementado con `socket` de MicroPython
- **Conexiones simultáneas:** Soporte para hasta 8 clientes

C. Control de Periféricos desde Pico W

- **Cámaras OV7670:** Control mediante I2C, captura mediante GPIOs paralelos
- **Motores:** Control PWM para velocidad y dirección
- **Servomotores:** Señales PWM para control de posición
- **Sensores:** Lectura mediante ADC y protocolos digitales

D. MicroPython en Pico W

Ventajas clave para este proyecto:

- Desarrollo rápido con sintaxis Python
- Soporte nativo para WiFi y sockets
- Librerías para control de hardware
- Recolección de basura para manejo de memoria

E. Configuración del Router

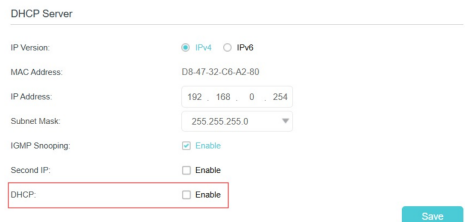


Fig. 1: Inhabilitación de DHCP

Servidor DHCP se inhabilita ya que la asignación de direcciones IP es estática

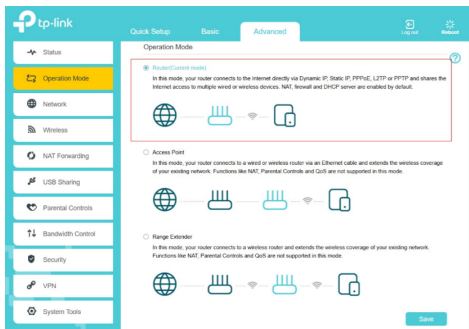


Fig. 2: Configuración como Router

(Modo Access point no sirvió ya que se requiere servidor HTTP para el envío de las imágenes)

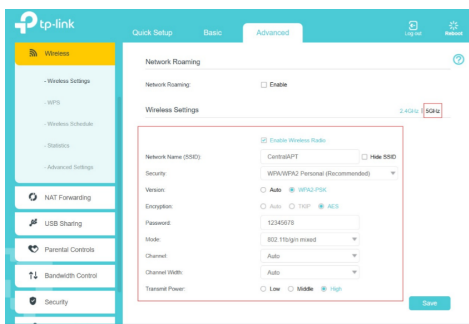


Fig. 3: Configuración De red

Configuración de Red. Solo se configura la red a 2.4GHz (banda soportada por Raspberry). La red 5GHz se apaga para evitar interferencia y no se utilizará.



Fig. 4: Configuración de la red LAN

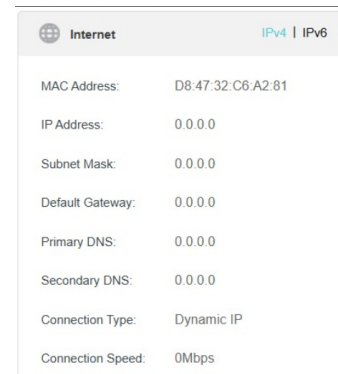


Fig. 5: Configuración de acceso a Internet se inhabilita, es innecesario

V. DISEÑO DEL SISTEMA

A. Arquitectura Hardware

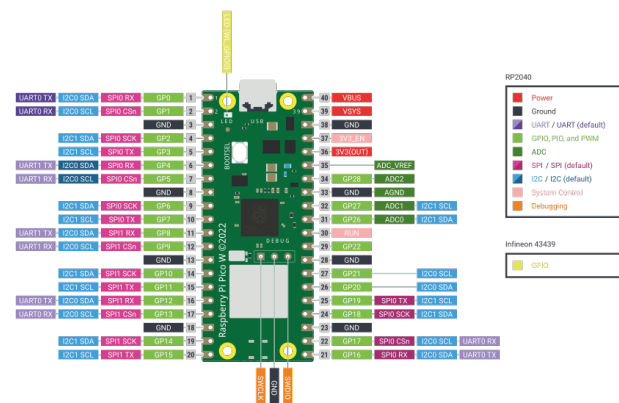


Fig. 6: Arquitectura del sistema implementado con Raspberry Pi Pico W

B. Configuración de Pines en Pico W

TABLE I: Configuración de pines en Raspberry Pi Pico W

Función	Pin Pico W	Descripción
UART TX	GPIO0	Transmisión serial a controlador principal
UART RX	GPIO1	Recepción serial desde controlador principal
I2C SDA	GPIO12	Comunicación con cámara OV7670
I2C SCL	GPIO13	Reloj I2C para cámara
MCLK	GPIO22	Reloj para cámara (generado con PWM)
PCLK	GPIO21	Pin de reloj de píxel de cámara
VSYNC	GPIO17	Sincronización vertical de cámara
HREF	GPIO26	Referencia horizontal de cámara
Datos cámara	GPIO2-9	Bus de datos paralelo de 8 bits
LED status	GPIO1	Indicador de estado

VI. DISEÑO

El sistema esta disenado par controlar a los WALLE de la siguiente manera:

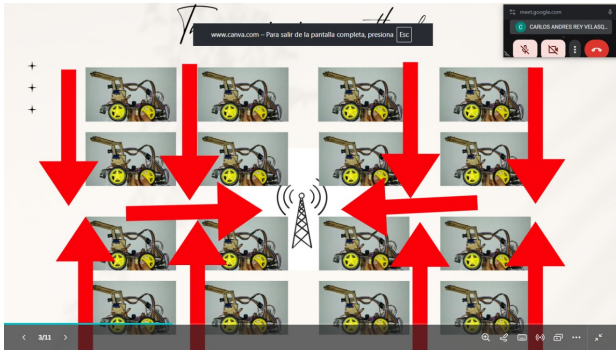


Fig. 7: Arquitectura general del sistema WALLI.

A continuación se describen los principales módulos implementados y sus funciones:

A. Central WiFi

ClaseCentralPoo.py: Crea un punto de acceso WiFi y escucha paquetes UDP. Recibe comandos JSON desde el computador y los reenvía a los carros.

```

1 import network
2 import socket
3 import json
4 import time
5 import select
6
7 class CentralWiFi:
8     def __init__(self, ssid, password, port=1234):
9         self.ssid = ssid
10        self.password = password
11        self.port = port
12        self.ap = network.WLAN(network.AP_IF)
13        self.socket = None
14        self._iniciar_ap()
15        self._iniciar_socket()
16
17    def _iniciar_ap(self):
18        self.ap.active(True)
19        self.ap.config(essid=self.ssid, password=
20        self.password)
21        while not self.ap.active():

```

```

        time.sleep(1)
        print("    AP activo:", self.ap.ifconfig())
24
25 def _iniciar_socket(self):
26     host = self.ap.ifconfig()[0]
27     self.socket = socket.socket(socket.AF_INET,
28     socket.SOCK_DGRAM)
29     self.socket.setsockopt(socket.SOL_SOCKET,
30     socket.SO_REUSEADDR, 1)
31     self.socket.bind((host, self.port))
32     self.socket.setblocking(False)
33     print(f"    Servidor UDP escuchando en {
34     host}:{self.port}")
35
36 def check_wifi(self):
37     if not self.ap.active():
38         print("    AP WiFi ca do, reactivando
39         ...")
40         self.ap.active(True)
41
42 def recibir_mensaje(self):
43     rlist, _, _ = select.select([self.socket],
44     [], [], 1)
45     if rlist:
46         try:
47             data, addr = self.socket.recvfrom
48             (4096)
49             if data:
50                 message = data.decode()
51                 print(f"    Recibido de {addr
52                 }: {message}")
53                 try:
54                     json_data = json.loads(
55                     message)
56                     print("    JSON recibido:",
57                     json_data)
58                     except Exception as e:
59                         print("    Error al parsear
60                         JSON:", e)
61                         json_data = {"error": "JSON
62                         inv lido"}
63                         return addr[0], addr[1],
64                         json_data
65                     except Exception as e:
66                         print("    Error recibiendo datos
67                         UDP:", e)
68                     return None, None, None
69
70 def enviar_json_a_dispositivo(self, ip_destino,
71     puerto_destino, json_data):
72     try:
73         mensaje = json.dumps(json_data)
74         self.socket.sendto(mensaje.encode(), (
75         ip_destino, puerto_destino))
76         print(f"    JSON enviado a {
77         ip_destino}:{puerto_destino}")
78     except Exception as e:
79         print("    Error al enviar JSON:", e)
80
81 def get_json_prueba(self):
82     return {
83         "mensaje": "Hola desde la central (
84         mensaje de prueba)",
85         "ip_destino": "192.168.4.60",
86         "ip_destino": "192.168.4.601",
87         "identifacor": "0000000",
88         "Carro_1": {
89             "Paso_1": {"Movimiento": {"
90             distancia_mm": 100, "
91             velocidad_mm_s": 50, "radio_mm":
92             "inf"}, "Brazo": {"
93             angulo0_grados": 0, "
94             angulo1_grados": 0, "
95             angulo2_grados": 0}},

```

```

72     "Paso_2": {"Movimiento": {"
        distancia_mm": 0, "
        velocidad_mm_s": 0, "radio_mm":
        0}, "Brazo": {"angulo0_grados":
        0, "angulo1_grados": 90, "
        angulo2_grados": 0}},
73     "Paso_3": {"Movimiento": {"
        distancia_mm": 0, "
        velocidad_mm_s": 0, "radio_mm":
        0}, "Brazo": {"angulo0_grados":
        0, "angulo1_grados": 0, "
        angulo2_grados": 90}},
74     "Paso_4": {"Movimiento": {"
        distancia_mm": 0, "
        velocidad_mm_s": 0, "radio_mm":
        0}, "Brazo": {"angulo0_grados":
        0, "angulo1_grados": 0, "
        angulo2_grados": -90}},
75     "Paso_5": {"Movimiento": {"
        distancia_mm": 0, "
        velocidad_mm_s": 0, "radio_mm":
        0}, "Brazo": {"angulo0_grados":
        0, "angulo1_grados": 0, "
        angulo2_grados": 90}},
76     "Paso_6": {"Movimiento": {"
        distancia_mm": 0, "
        velocidad_mm_s": 0, "radio_mm":
        0}, "Brazo": {"angulo0_grados":
        0, "angulo1_grados": 0, "
        angulo2_grados": -90}},
77     "Paso_7": {"Movimiento": {"
        distancia_mm": 0, "
        velocidad_mm_s": 0, "radio_mm":
        0}, "Brazo": {"angulo0_grados":
        0, "angulo1_grados": -90, "
        angulo2_grados": 0}},
78     "Paso_8": {"Movimiento": {"
        distancia_mm": 100, "
        velocidad_mm_s": 50, "radio_mm":
        50}, "Brazo": {"angulo0_grados":
        -90, "angulo1_grados": 0, "
        angulo2_grados": 0}}
79 },
80 "timestamp": time.time(),
81 "status": "ok"
82 }

```

Listing 1: ClaseCentralPoo.py

El código implementa una clase llamada CentralWiFi que actúa como una central que reparte señal WiFi mediante un *Access Point* (AP) y se comunica con dispositivos conectados utilizando el protocolo UDP para enviar y recibir datos en formato JSON. A continuación, se describen los elementos principales:

- **Importación de módulos:** Se utilizan los módulos `network` para la configuración del AP, `socket` para la comunicación UDP, `json` para codificar y decodificar mensajes, `time` para pausas y marcas de tiempo, y `select` para la verificación de datos disponibles de forma no bloqueante.
- **Constructor (`__init__`):** Configura el SSID, la contraseña y el puerto del AP, creando una instancia del AP y un socket UDP no bloqueante. Llama a métodos internos para iniciar el AP y el socket de servidor.
- **Método `iniciar_ap`:** Activa el AP y configura su SSID y contraseña. Espera hasta que el AP esté activo antes de continuar y muestra en consola la configuración IP.

- **Método `iniciar_socket`:** Obtiene la IP del AP y configura un socket UDP no bloqueante que escucha en el puerto indicado, permitiendo la reutilización del puerto con `SO_REUSEADDR`.
- **Método `check_wifi`:** Verifica si el AP sigue activo. Si se detecta caída, se reactiva automáticamente.
- **Método `recibir_mensaje`:** Utiliza `select` para comprobar si hay datos entrantes en el socket. Si los hay, se reciben y decodifican en formato UTF-8. Intenta decodificar el mensaje como JSON, y si se logra, retorna la IP, puerto del remitente y los datos en JSON. En caso de error, retorna un diccionario de error.
- **Método `enviar_json_a_dispositivo`:** Permite enviar un objeto JSON codificado como cadena a una IP y puerto destino mediante el socket UDP, mostrando en consola confirmación o error.
- **Método `get_json_prueba`:** Retorna un objeto de prueba en formato JSON que incluye un mensaje de prueba, direcciones IP de destino, identificador y una estructura de pasos con parámetros de movimientos y ángulos de un brazo robótico, junto con una marca de tiempo y un estado de confirmación.

Este diseño permite:

- Crear un AP WiFi para que dispositivos (por ejemplo, robots móviles) se conecten sin depender de un router externo.
- Recibir comandos en formato JSON de forma no bloqueante mediante UDP.
- Enviar datos de control o confirmación en formato JSON a dispositivos conectados.
- Utilizar una estructura modular y clara para integrar con controladores de robots (por ejemplo, Raspberry Pi Pico W) de forma escalable y organizada.

B. Servidor de Imágenes

`Servidor_Imagenes.py`: Servidor Flask que recibe imágenes en formato RGB565, las convierte a RGB888 y las guarda en JPG. Identifica las imágenes por IP y secuencia.

```

1 from flask import Flask, request, jsonify
2 import os
3 import threading
4 from datetime import datetime
5 import cv2
6 import numpy as np
7 from collections import defaultdict
8 import json
9
10 app = Flask(__name__)
11
12 # Directories for saving images
13 IMAGE_DIR = r"C:\Users\Public\Downloads\ov7670\
    imagenes" # Not used directly for saving in this
    version
14 PROCESSED_DIR = r"C:\Users\Public\Downloads\ov7670\
    procesadas"
15
16 # Create directories if they don't exist
17 os.makedirs(IMAGE_DIR, exist_ok=True)
18 os.makedirs(PROCESSED_DIR, exist_ok=True)
19

```

```

20 # Global dictionary to store image counters for each
    IP address (as a fallback or for other clients)
21 image_counters = defaultdict(int)
22 # File to persist counters, so they don't reset on
    server restart
23 COUNTERS_FILE = os.path.join(PROCESSED_DIR, "
    server_image_counters.json")
24
25 # Lock for thread-safe access to image_counters (if
    fallback is used)
26 lock = threading.Lock()
27
28 # Load existing counters on server startup
29 if os.path.exists(COUNTERS_FILE):
30     try:
31         with open(COUNTERS_FILE, 'r') as f:
32             loaded_counters = json.load(f)
33             for k, v in loaded_counters.items():
34                 image_counters[k] = v
35             print(f"Contadores cargados: {dict(
                image_counters)}")
36     except json.JSONDecodeError:
37         print("Warning: Counters file corrupted or
            empty. Starting counters from scratch.")
38     except Exception as e:
39         print(f"Error loading counters: {e}")
40
41 # --- Image Processing Functions ---
42
43 def rgb565_to_rgb888(rgb565_bytes):
44     """
45     Converts 16-bit RGB565 raw image bytes to 24-bit
        RGB888.
46     Output is in BGR format, which is standard for
        OpenCV.
47     """
48     result = bytearray()
49     for i in range(0, len(rgb565_bytes), 2):
50         pixel = (rgb565_bytes[i] << 8) |
            rgb565_bytes[i+1]
51         r = (pixel >> 11) & 0x1F
52         g = (pixel >> 5) & 0x3F
53         b = pixel & 0x1F
54
55         r = (r << 3) | (r >> 2)
56         g = (g << 2) | (g >> 4)
57         b = (b << 3) | (b >> 2)
58
59         result.extend([b, g, r])
60     return bytes(result)
61
62 def process_and_save_image(width, height,
    rgb888_data, device_ip,
    sequence_number_from_client=None):
63     """
64     Processes RGB888 image data, saves it as a JPG
        file,
65     and names it using the device's IP and either
        its provided sequence number
66     or a server-generated sequential number.
67     """
68     print(f"[    Processing and Saving] Image
        from {width}x{height} from {device_ip}")
69
70     np_image = np.frombuffer(rgb888_data, dtype=np.
        uint8).reshape((height, width, 3))
71
72     # Determine the sequence number to use for the
        filename
73     current_sequence_number =
        sequence_number_from_client
74     if current_sequence_number is None:
75         # If no sequence number from client, use
            server's internal counter (and increment
            it)
76         with lock:
77             image_counters[device_ip] += 1
78             current_sequence_number = image_counters
                [device_ip]
79             # Save updated counters to file
80             try:
81                 with open(COUNTERS_FILE, 'w') as f:
82                     json.dump(dict(image_counters),
                        f)
83             except Exception as e:
84                 print(f"    Error saving counters: {
                    e}")
85             print(f"    No sequence number from
                client, using server-generated: {
                    current_sequence_number}")
86         else:
87             print(f"    Using client-provided
                sequence number: {
                    current_sequence_number}")
88
89     # Generate filename using sanitized IP and the
        determined sequence number
90     sanitized_ip = device_ip.replace('.', '-')
91     timestamp = datetime.now().strftime("%Y%m%d_%H%M
        %S")
92
93     # Construct the full filename (e.g.,
        192-168-0-200_0001_YYYYMMDD_HHMMSS.jpg)
94     processed_filename_jpg = f"{sanitized_ip}_{int(
        current_sequence_number):04d}_{timestamp}.
        jpg"
95     processed_path = os.path.join(PROCESSED_DIR,
        processed_filename_jpg)
96
97     # Save the processed image (JPG) using OpenCV
98     try:
99         success = cv2.imwrite(processed_path,
            np_image)
100         if not success:
101             print(f"[    Error] Could not save
                processed image to {processed_path}")
102             return
103         print(f"[    Image processed] Saved to: {
            processed_path}")
104     except Exception as e:
105         print(f"[    Error] Failed to save image {
            processed_path}: {e}")
106
107 # --- Flask App Endpoints ---
108
109 @app.route("/upload_raw_image_flash/", methods=["
    POST"])
110 def upload_image():
111     """
112     Endpoint to receive raw image data from the
        Raspberry Pi Pico W.
113     It expects the device's IP in the 'X-Device-IP'
        header and
114     the sequence number in the 'X-Image-Sequence'
        header.
115     """
116     device_ip = request.headers.get("X-Device-IP",
        request.remote_addr)
117     # Get sequence number from header; convert to
        int. If not present, it's None.
118     image_sequence = request.headers.get("X-Image-
        Sequence")
119     if image_sequence:
120         try:
121             image_sequence = int(image_sequence)
122         except ValueError:
123             print(f"[    Warning] Invalid X-Image

```

```

124         -Sequence header: {image_sequence}.
           Treating as None.")
125         image_sequence = None # Fallback if
           header is present but not a valid
           integer
126
127     print(f"        Image received from: {device_ip},
           Sequence: {image_sequence} if image_sequence
           is not None else 'N/A'")
128
129     data = request.get_data()
130
131     if len(data) < 4:
132         print("[    Error] Insufficient data
           received.")
133         return jsonify({"status": "error", "message":
           "Insuficiente data."}), 400
134
135     width = int.from_bytes(data[0:2], 'big')
136     height = int.from_bytes(data[2:4], 'big')
137     image_data_rgb565 = data[4:]
138
139     expected_size = width * height * 2
140     if len(image_data_rgb565) != expected_size:
141         print(f"[    Error] Invalid image data size.
           Expected: {expected_size}, Received: {
           len(image_data_rgb565)}")
142         return jsonify({"status": "error", "message":
           f"Tama o de datos inv lido. Esperado
           {expected_size} bytes, recibido {len(
           image_data_rgb565)} bytes."}), 400
143
144     rgb888_data = rgb565_to_rgb888(image_data_rgb565
           )
145
146     # Pass the client-provided sequence number to
           the processing function
147     threading.Thread(target=process_and_save_image,
           args=(width, height,
           rgb888_data, device_ip,
           image_sequence),
           daemon=True).start()
148
149     return jsonify({
150         "status": "ok",
151         "device_ip": device_ip,
152         "sequence_number": image_sequence,
153         "message": "Imagen recibida y procesamiento
           iniciado en segundo plano."
154     })
155
156 if __name__ == "__main__":
157     print(f"Starting Flask server on http
           ://0.0.0.0:8000")
158     print(f"Processed images (JPG) will be saved in:
           {PROCESSED_DIR}")
159     app.run(host="0.0.0.0", port=8000, debug=False,
           threaded=True)
160

```

Listing 2: Servidor_Imagenes.py

El código implementa un servidor Flask que permite la recepción de imágenes en crudo enviadas desde dispositivos como la Raspberry Pi Pico W con cámara OV7670, procesándolas y almacenándolas en el equipo servidor. A continuación, se describen sus partes principales:

- **Importación de librerías:** Se importan librerías necesarias para la construcción del servidor, manejo de imágenes, procesamiento de datos en segundo plano y manejo de estructuras de datos:
 - Flask: para construir el servidor HTTP.

- os, threading, datetime, json: para gestión de archivos, multihilos, fechas y almacenamiento de datos.
- cv2, numpy: para procesar y guardar imágenes.
- defaultdict: para manejar contadores de imágenes de forma sencilla.

- **Estructura de carpetas:** Se crean y gestionan carpetas para almacenar las imágenes procesadas, asegurando su existencia antes de iniciar el servidor.
- **Persistencia de contadores:** Se mantiene un diccionario global con contadores de imágenes recibidas por cada dirección IP de los dispositivos conectados, utilizando un archivo JSON para persistencia, de modo que los contadores no se reinicien al reiniciar el servidor.
- **Conversión RGB565 a RGB888:** Se implementa una función que convierte los bytes de imagen en formato RGB565 (16 bits por píxel) a RGB888 (24 bits por píxel) en el orden BGR que usa OpenCV, permitiendo así guardar las imágenes en un formato visualizable.
- **Procesamiento y guardado de imágenes:** Se implementa una función que:
 - Convierte los datos a una matriz NumPy con la resolución recibida.
 - Determina un número de secuencia para el nombre del archivo, ya sea proporcionado por el cliente o generado por el servidor.
 - Genera un nombre de archivo con la IP del dispositivo, el número de secuencia y la marca de tiempo.
 - Guarda la imagen en formato JPG utilizando OpenCV en la carpeta designada.
- **Punto de acceso (endpoint) /upload_raw_image_flash/:** Este endpoint:
 - Recibe solicitudes POST con la imagen en crudo.
 - Obtiene la IP del dispositivo desde las cabeceras y, opcionalmente, el número de secuencia.
 - Valida el tamaño de los datos y los separa en ancho, alto y datos de imagen.
 - Convierte los datos RGB565 a RGB888.
 - Procesa y guarda la imagen en un hilo en segundo plano para no bloquear el servidor.
 - Devuelve un JSON de confirmación con la IP, número de secuencia y estado.
- **Ejecución del servidor:** El servidor se ejecuta escuchando en la dirección 0.0.0.0 en el puerto 8000, permitiendo accesos desde la red local.

Este diseño permite:

- Recibir imágenes en crudo de dispositivos de bajo costo como la Raspberry Pi Pico W con cámaras OV7670 en tiempo real.
- Procesar las imágenes automáticamente y guardarlas en el servidor con un esquema organizado por IP, secuencia y tiempo.
- Mantener un control de los números de secuencia de imágenes de cada dispositivo, permitiendo retomar el conteo en caso de reinicio del servidor.

- Procesar imágenes en segundo plano sin detener la recepción de nuevas imágenes.

C. Carro como Cliente WiFi

carro_wifi_module.py: El carro se conecta a la red WiFi, escucha comandos UDP y los reenvía por UART a su microcontrolador principal.

```

1 # carro_wifi_module.py
2
3 import machine
4 import time
5 import network
6 import socket
7 import json
8
9 class CarroWiFi:
10     def __init__(self, ssid, password, host, port,
11                  local_port, my_ip, uart_tx, uart_rx,
12                  uart_baud=9600):
13         # UART
14         self.uart = machine.UART(0, baudrate=
15             uart_baud, tx=machine.Pin(uart_tx), rx=
16             machine.Pin(uart_rx))
17
18         # WiFi
19         self.ssid = ssid
20         self.password = password
21         self.host = host
22         self.port = port
23         self.local_port = local_port
24         self.my_ip = my_ip
25
26         self.wlan = network.WLAN(network.STA_IF)
27         self.wlan.ifconfig((self.my_ip, '
28             255.255.255.0', self.host, '8.8.8.8'))
29
30         # Socket UDP
31         self.s = socket.socket(socket.AF_INET,
32             socket.SOCK_DGRAM)
33         self.s.bind(('0.0.0.0', self.local_port))
34         self.s.setblocking(False)
35
36         # Buffer UART
37         self.buffer = b""
38         self.last_send = 0
39         self.send_interval = 5
40
41         # Conectar al iniciar
42         self.connect_wifi()
43
44     def connect_wifi(self):
45         self.wlan.active(True)
46         if not self.wlan.isconnected():
47             print("Conectando a WiFi...")
48             self.wlan.connect(self.ssid, self.
49                 password)
50             timeout = 10
51             start = time.time()
52             while not self.wlan.isconnected():
53                 if time.time() - start > timeout:
54                     print("Timeout conectando
55                         WiFi")
56                     break
57                 time.sleep(1)
58             if self.wlan.isconnected():
59                 print("WiFi conectado:", self.wlan.
60                     ifconfig())
61             else:
62                 print("No conectado a WiFi")
63
64     def ensure_wifi(self):
65         if not self.wlan.isconnected():

```

```

66         print("WiFi desconectado, intentando
67             reconectar...")
68         self.connect_wifi()
69
70     def read_internal_temp(self):
71         sensor_temp = machine.ADC(4)
72         conversion_factor = 3.3 / 65535
73         reading = sensor_temp.read_u16() *
74             conversion_factor
75         temperature = 27 - (reading - 0.706) /
76             0.001721
77         return round(temperature, 2)
78
79     def send_json(self, data):
80         try:
81             self.ensure_wifi()
82             if not self.wlan.isconnected():
83                 print("No conectado a WiFi, no se
84                     envia")
85                 return
86
87             json_str = json.dumps(data)
88             self.s.sendto(json_str.encode(), (self.
89                 host, self.port))
90         except Exception as e:
91             print("Error al enviar al servidor:",
92                 e)
93
94     def recibir_del_central(self):
95         try:
96             data, addr = self.s.recvfrom(4096)
97             msg = data.decode('utf-8')
98             try:
99                 json_data = json.loads(msg)
100                 ip_destino = json_data.get("
101                     ip_destino", "")
102                 if ip_destino == self.my_ip:
103                     # Aqu puedes procesar el
104                     # mensaje localmente si lo
105                     # deseas
106                     pass
107                 else:
108                     self.uart.write(json.dumps(
109                         json_data) + "\n")
110                 return msg
111             except json.JSONDecodeError:
112                 return msg # Retornar aunque sea
113                     malformado
114         except OSError:
115             return None

```

Listing 3: carro_wifi_module.py

Este código implementa una clase llamada CarroWiFi diseñada para ser ejecutada en dispositivos embebidos (como la Raspberry Pi Pico W), permitiendo el control de un carro móvil con comunicación WiFi mediante UDP, manejo de comunicación con un microcontrolador de control del carro vía UART y lectura de temperatura interna del microcontrolador.

A continuación, se describen sus elementos principales:

- **Importación de librerías:** Se utilizan las librerías machine (control de hardware y pines), time (gestión de tiempos), network (manejo de WiFi), socket (UDP) y json (codificación y decodificación de datos).
- **Constructor (__init__):** Configura:
 - **UART:** para comunicarse con otro microcontrolador que maneja el movimiento y sensores del carro.
 - **WiFi:** define el SSID, contraseña y configura una IP estática para el carro, la puerta de enlace (gateway)

y DNS.

- **Socket UDP:** para enviar y recibir datos en la red local, utilizando un puerto local y uno de destino en el servidor central.
- **Buffer y temporizador:** para manejar intervalos de envío.
- Llama a `connect_wifi` para conectarse a la red WiFi al inicializarse.

- **Método `connect_wifi`:** Activa la interfaz WiFi y gestiona la conexión a la red definida, con un tiempo de espera de 10 segundos antes de abortar el intento.
- **Método `ensure_wifi`:** Verifica de manera periódica si el dispositivo está conectado a la red WiFi y, en caso contrario, intenta reconectarse automáticamente.
- **Método `read_internal_temp`:** Permite la lectura de la temperatura interna del microcontrolador utilizando el ADC del canal interno, retornando la temperatura en grados Celsius.
- **Método `send_json`:** Permite enviar datos al servidor central en formato JSON a través del socket UDP, asegurándose previamente de que exista conexión WiFi.
- **Método `recibir_del_central`:** Permite recibir datos enviados desde el servidor central a través del socket UDP. Intenta decodificar el mensaje recibido como JSON:
 - Si el campo `"ip_destino"` coincide con la IP local del carro, se puede procesar localmente.
 - Si no coincide, el mensaje es reenviado al microcontrolador del carro a través de UART para su interpretación y acción.

En caso de que el mensaje esté mal formado, se retorna el texto recibido aunque no sea JSON válido.

Este módulo permite:

- Crear un sistema de comunicación bidireccional entre un carro móvil con la Raspberry Pi Pico W y un servidor central usando UDP y WiFi.
- Controlar el carro a través de comandos enviados desde el servidor central.
- Leer la temperatura interna para monitoreo de condiciones operativas.
- Reenviar datos o comandos vía UART a un microcontrolador que controle motores y sensores del carro.

D. Carro como AP y Repetidor UART

`carroAPpoo.py`: El carro funciona como AP y como nodo intermedio. Recibe datos UDP o UART y los reenvía o ejecuta según la IP destino.

```
1 # clase_carro_ap.py
2 import machine
3 import network
4 import socket
5 import json
6 import time
7
8 class CarroAP:
9     def __init__(self, ssid="carro_ap", password="
10         12345678", udp_port=1234,
```

```
        forward_ip="192.168.4.17",
        forward_port=5555, my_ipint="
192.168.4.123"):
```

```
    self.ssid = ssid
    self.password = password
    self.udp_port = udp_port
    self.forward_ip = forward_ip
    self.forward_port = forward_port
    self.my_ipint = my_ipint

    # UART
    self.uart = machine.UART(0, baudrate=9600,
        tx=machine.Pin(0), rx=machine.Pin(1))
    self.uart_buffer = b""

    # ADC para temperatura
    self.adc = machine.ADC(4)

    # AP
    self.ap = network.WLAN(network.AP_IF)
    self.ap.active(True)
    self.ap.config(essid=self.ssid, password=
        self.password)

    while not self.ap.active():
        pass

    self.my_ip = self.ap.ifconfig()[0]
    print("    AP activo:", self.ap.ifconfig())

    # Socket UDP
    self.udp_sock = socket.socket(socket.AF_INET
        , socket.SOCK_DGRAM)
    self.udp_sock.setblocking(False)
    self.udp_sock.bind((self.my_ip, self.
        udp_port))

    # Temporizador
    self.last_send = 0
    self.send_interval = 5 # segundos

    def read_temp(self):
        voltage = self.adc.read_u16() * 3.3 / 65535
        return round(27 - (voltage - 0.706) /
            0.001721, 2)

    def enviar_temp_uart(self):
        now = time.time()
        if now - self.last_send >= self.
            send_interval:
            temp = self.read_temp()
            data = {"temp_carro_ap": temp}
            self.uart.write(json.dumps(data) + "\n")
            print("    UART (temp):", data)
            self.last_send = now

    def escuchar_udp(self):
        try:
            data, addr = self.udp_sock.recvfrom
                (1024)
            msg = data.decode()
            print("    UDP de", addr, ":", msg)
            try:
                json_data = json.loads(msg)
                self.uart.write(json.dumps(json_data
                    ) + "\n")
                print("    Reenviado por UART:",
                    json_data)
            except Exception as e:
                print("    JSON inv lido:", e)
                json_data = {"error": "JSON
                    inv lido"}
```

```

72     response = {"status": "ok", "received":
73                 json_data}
74     self.udp_sock.sendto(json.dumps(response)
75                          ).encode(), addr)
76
77 except OSError:
78     pass # No hay datos
79
80 def escuchar_uart(self):
81     if self.udp_sock.any():
82         self.udp_sock.recv_into(self.udp_buffer)
83         if b"\n" in self.udp_buffer:
84             line, self.udp_buffer = self.
85                 udp_buffer.split(b"\n", 1)
86             try:
87                 received = json.loads(line.
88                                     decode())
89                 ip_destino = received.get("
90                                     ip_destino", "")
91
92                 if ip_destino == self.my_ipint:
93                     print("      UART recibido
94                           (para m ): ", received)
95                     carro_1 = received.get("
96                           Carro_1", {})
97                     for paso, datos in carro_1.
98                         items():
99                         mov = datos.get("
100                               Movimiento", {})
101                         brazo = datos.get("Brazo
102                               ", {})
103                     print(f"paso:
104                           Movimiento = {mov},
105                           Brazo = {brazo}")
106                 else:
107                     self.udp_sock.sendto(json.
108                                         dumps(received).encode()
109                                         ,
110                                         (self.
111                                         forward_ip
112                                         ,
113                                         self.
114                                         .
115                                         forward_port
116                                         ))
117
118                     print(f"      UART
119                           redirigido a {self.
120                           forward_ip}:{self.
121                           forward_port}: ",
122                           received)
123
124 except Exception as e:
125     print("      Error al
126           decodificar UART:", e)
127
128 def ciclo_principal(self):
129     while True:
130         self.enviar_temp_uart()
131         self.escuchar_udp()
132         self.escuchar_uart()
133         time.sleep(0.1)

```

Listing 4: carroAppoo.py

Este código implementa una clase llamada CarroAP para la Raspberry Pi Pico W, configurando un Access Point (AP) propio, comunicación UDP, UART con otro microcontrolador y monitoreo de temperatura, permitiendo operar un carro móvil sin necesidad de un router externo.

A continuación, se detallan sus partes principales:

- **Importación de librerías:** Se utilizan:

- machine: manejo de pines, UART y ADC para temperatura.
- network: creación de Access Point.
- socket: comunicación UDP.
- json, time: manejo de datos y temporización.

- **Constructor (__init__):** Inicializa:

- **Parámetros de AP:** nombre de red (ssid), contraseña y puerto UDP de escucha.
- **Parámetros de reenvío UDP:** IP y puerto destino para reenvío de mensajes si no están dirigidos al dispositivo actual.
- **UART:** configuración de comunicación con otro microcontrolador a 9600 baudios.
- **ADC:** para lectura de temperatura interna.
- **Access Point:** activa y configura el AP, esperando hasta que esté activo y mostrando la IP asignada.
- **Socket UDP:** configura un socket no bloqueante que escucha en la IP del AP y el puerto UDP indicado.
- Configuración de temporización para el envío periódico de temperatura.

- **Método read_temp:** Lee la temperatura interna utilizando el ADC, calculando y devolviendo la temperatura en grados Celsius.

- **Método enviar_temp_uart:** Envía periódicamente la temperatura leída a través del UART en formato JSON al microcontrolador que controla el carro, mostrando en consola el dato enviado.

- **Método escuchar_udp:**

- Escucha mensajes UDP recibidos.
- Intenta decodificar los datos en JSON.
- Reenvía el JSON recibido por UART al microcontrolador de control.
- Envía al cliente una respuesta de confirmación con el JSON recibido o un error en caso de datos inválidos.

- **Método escuchar_uart:**

- Lee datos desde el UART si hay disponibles, acumulándolos hasta detectar un salto de línea.
- Decodifica la línea recibida en JSON.
- Verifica si el campo "ip_destino" coincide con la IP interna del dispositivo:

- * Si coincide, interpreta el mensaje como comando local para el carro, mostrando en consola los pasos y movimientos a ejecutar.

- * Si no coincide, reenvía el JSON a través de UDP al destino configurado, permitiendo actuar como repetidor o nodo de retransmisión.

- **Método ciclo_principal:** Ejecuta un ciclo infinito que:

- Envía la temperatura periódicamente por UART.
- Escucha y procesa mensajes UDP.
- Escucha y procesa mensajes recibidos por UART.
- Incorpora una pequeña espera para estabilidad del bucle.

Este módulo permite:

- Configurar el carro móvil como un **Access Point autónomo**, eliminando la necesidad de infraestructura WiFi externa.
- Recibir comandos o configuraciones por UDP y retransmitirlos al microcontrolador de control del carro vía UART.
- Recibir comandos por UART y decidir localmente si procesarlos o reenviarlos a otro nodo vía UDP.
- Monitorear y reportar la temperatura interna del microcontrolador, útil para sistemas de protección térmica.
- Actuar como nodo repetidor de comandos entre carros, permitiendo sistemas de flotas y colaboración entre múltiples robots móviles.

E. Captura de Imágenes con OV7670

Recibir_Imagenes.py: Captura imágenes usando la cámara OV7670 y las envía en formato binario al servidor Flask, incluyendo un número de secuencia.

```

1 #Recibir_Imagenes.py
2 import machine
3 import time
4 import urequests as requests
5 import sys
6 import ujson
7 from machine import Pin, I2C, PWM
8 from ov7670_wrapper import * # Assuming this is your
   custom camera wrapper
9 import gc
10 import network
11
12 # Global counter for image sequence
13 image_sequence_number = 0
14
15 def conectar_wifi_estatico(ssid, password,
16 ip_estatica, mascara_subred, puerta_enlace,
17 dns_primario, dns_secundario=None):
18     wlan = network.WLAN(network.STA_IF)
19     wlan.active(True)
20     if not wlan.isconnected():
21         print('Conectando a la red Wi-Fi con IP
22         estatica...')
23         wlan.ifconfig((ip_estatica, mascara_subred,
24             puerta_enlace, dns_primario))
25
26         wlan.connect(ssid, password)
27         timeout = 20
28         while not wlan.isconnected() and timeout >
29             0:
30             print('.', end='')
31             time.sleep(1)
32             timeout -= 1
33
34     if wlan.isconnected():
35         print('\nConectado a Wi-Fi.')
36         print(f'IP: {wlan.ifconfig()[0]}')
37         print(f'Mscara de subred: {wlan.ifconfig()
38             [1]}')
39         print(f'Puerta de enlace: {wlan.ifconfig()
40             [2]}')
41         print(f'DNS: {wlan.ifconfig()[3]}')
42         return True
43     else:
44         print('\nError: no se pudo conectar a Wi-Fi')
45         )

```

```

38         return False
39
40 # --- Network Data (ADJUST THESE VALUES TO YOUR
   NETWORK!) ---
41 SSID = "CentralAPT"
42 PASSWORD = "12345678"
43 IP_ESTATICA = "192.168.0.200" # The IP you want
   your microcontroller to have
44 MASCARA_SUBRED = "255.255.255.0"
45 PUERTA_ENLACE = "192.168.0.254" # Your router's
   IP or gateway
46 DNS_PRIMARIO = "8.8.8.8" # A public DNS server
   (Google DNS)
47 DNS_SECUNDARIO = "8.8.4.4" # Another public DNS
   server
48
49 # The IP of the server where you'll send images
50 FLASH_SERVER_URL = "http://192.168.0.100:8000"
51
52 if not conectar_wifi_estatico(SSID, PASSWORD,
53 IP_ESTATICA, MASCARA_SUBRED, PUERTA_ENLACE,
54 DNS_PRIMARIO, DNS_SECUNDARIO):
55     print("No conectado a Wi-Fi, abortando...")
56     sys.exit()
57
58 # --- Rest of the code (significant changes here for
   sequence number) ---
59
60 UPLOAD_ENDPOINT = "/upload_raw_image_flash/"
61 TIMEOUT_SECONDS = 30
62 RETRY_ATTEMPTS = 3
63 RETRY_DELAY_SECONDS = 5
64 DEVICE_IP = IP_ESTATICA
65
66 # Camera pins
67 mclk_pin_no = 22
68 pclk_pin_no = 21
69 data_pin_base = 2
70 vsync_pin_no = 17
71 href_pin_no = 26
72 reset_pin_no = 14
73 shutdown_pin_no = 15
74 sda_pin_no = 12
75 scl_pin_no = 13
76 led_pin = Pin(1, Pin.OUT)
77
78 # PWM for MCLK
79 print("Configuring MCLK for OV7670 on GP22...")
80 pwm = PWM(Pin(mclk_pin_no))
81 pwm.freq(30_000_000)
82 pwm.duty_u16(32768)
83
84 # Initialize I2C and camera
85 print("Initializing I2C and OV7670...")
86 i2c = I2C(0, freq=400_000, scl=Pin(scl_pin_no), sda=
87     Pin(sda_pin_no))
88
89 try:
90     ov7670 = OV7670Wrapper(
91         i2c_bus=i2c,
92         mclk_pin_no=mclk_pin_no,
93         pclk_pin_no=pclk_pin_no,
94         data_pin_base=data_pin_base,
95         vsync_pin_no=vsync_pin_no,
96         href_pin_no=href_pin_no,
97         reset_pin_no=reset_pin_no,
98         shutdown_pin_no=shutdown_pin_no,
99     )
100     ov7670.wrapper_configure_rgb()
101     ov7670.wrapper_configure_base()
102     width, height = ov7670.wrapper_configure_size(
103         OV7670_WRAPPER_SIZE_DIV4)
104     i2c.writeto_mem(0x21, 0x13, b'\xE7')
105     ov7670.wrapper_configure_test_pattern()

```

```

OV7670_WRAPPER_TEST_PATTERN_NONE)
102
103 print(f"    OV7670 initialized. Resolution: {
    width}x{height}")
104 frame_buf = bytearray(width * height * 2)
105 gc.collect()
106 except Exception as e:
107     print(f"    Error initializing camera: {e}")
108     sys.exit(1)
109
110 # Modified send_image_to_flash to include sequence
    number
111 def send_image_to_flash(image_data_buffer, img_width
    , img_height, seq_num):
112     UPLOAD_URL = f"{FLASH_SERVER_URL}{
        UPLOAD_ENDPOINT}"
113
114     full_raw_image_data = img_width.to_bytes(2, 'big
        ') + \
115                             img_height.to_bytes(2, '
        big') + \
116                             image_data_buffer
117
118     headers = {
119         "Content-Type": "application/octet-stream",
120         "X-Device-IP": DEVICE_IP,
121         "X-Image-Sequence": str(seq_num) # Add the
            sequence number here
122     }
123
124     for attempt in range(RETRY_ATTEMPTS):
125         print(f"        Sending {len(
            full_raw_image_data)} bytes (Seq: {
            seq_num}) to {UPLOAD_URL}... (Attempt {
            attempt + 1}/{RETRY_ATTEMPTS})")
126         try:
127             response = requests.post(UPLOAD_URL,
                data=full_raw_image_data, headers=
                headers, timeout=TIMEOUT_SECONDS)
128             if response.status_code == 200 or
                response.status_code == 201:
129                 try:
130                     json_data = ujson.loads(response
                        .text)
131                 except Exception as e:
132                     print(f"        Error parsing JSON
                        response: {e}")
133                     response.close()
134                     return None
135                     response.close()
136                     return json_data
137             else:
138                 print(f"        Error sending. HTTP Code
                    : {response.status_code}")
139                 response.close()
140                 if attempt < RETRY_ATTEMPTS - 1:
141                     time.sleep(RETRY_DELAY_SECONDS)
142                 else:
143                     return None
144         except Exception as e:
145             print(f"        Connection error: {e}")
146             if attempt < RETRY_ATTEMPTS - 1:
147                 time.sleep(RETRY_DELAY_SECONDS)
148             else:
149                 return None
150     return None
151
152 # Continuous capture and send
153 if __name__ == "__main__":
154     gc.collect()
155     while True:
156         print("\n--- Capturing image ---")
157         ov7670.capture(frame_buf)
158         print("    Image captured.")
159
160     # Increment sequence number for each capture
    image_sequence_number += 1
161     print(f"    Assigning sequence number: {
        image_sequence_number}")
162
163     # Pass the sequence number to the sending
    function
164     resp = send_image_to_flash(frame_buf, width,
        height, image_sequence_number)
165     if resp:
166         print(f"    Server response: {resp}")
167     else:
168         print("    Could not send image.")
169
170     time.sleep(2)

```

Listing 5: Recibir_Imagenes.py

Este código implementa un sistema completo para **capturar imágenes con la cámara OV7670** usando la **Raspberry Pi Pico W** y enviarlas a un servidor Flask a través de HTTP POST, utilizando IP estática y WiFi, con manejo de secuencias para seguimiento de imágenes enviadas.

A continuación, se explican las secciones principales:

- **Importación de librerías:** Se utilizan:
 - machine, Pin, I2C, PWM para manejo de hardware.
 - network para conexión WiFi.
 - urequests para enviar datos al servidor.
 - ujson para manejar datos en JSON.
 - gc para manejo de memoria.
 - ov7670_wrapper para controlar la cámara OV7670 de manera simplificada.
- **Conexión a WiFi con IP estática:** Se define la función `conectar_wifi_estatico` para conectarse a una red WiFi utilizando una dirección IP fija, junto con máscara de subred, puerta de enlace y DNS, mostrando información de conexión en consola para verificación.
- **Parámetros de red y servidor:** Se configuran:
 - SSID, contraseña y parámetros de IP estática de la red.
 - IP y puerto del servidor Flask donde se enviarán las imágenes.
- **Inicialización de la cámara OV7670:**
 - Se configuran pines para el reloj (MCLK), datos, VSYNC y HREF.
 - Se usa PWM para generar el MCLK necesario para la cámara.
 - Se inicializa el bus I2C a 400 kHz.
 - Se configura la cámara usando el wrapper, seleccionando el modo RGB, tamaño de imagen y patrones de prueba.
 - Se crea un buffer `frame_buf` para almacenar cada imagen capturada.
- **Función `send_image_to_flash`:** Envía la imagen capturada al servidor Flask usando HTTP POST:
 - Prepara la imagen agregando el ancho, alto y los datos de la imagen en bruto.

- Incluye en las cabeceras el tipo de contenido (application/octet-stream), la IP del dispositivo y el número de secuencia de la imagen.
 - Realiza múltiples intentos en caso de error, con reintentos y esperas entre cada intento.
 - Si la respuesta es exitosa, intenta decodificar el JSON de confirmación del servidor y lo retorna.
- **Bucle principal:** Si el script se ejecuta como principal:
 - Captura continuamente imágenes utilizando `ov7670.capture`.
 - Incrementa el número de secuencia en cada captura para un control de imágenes enviadas.
 - Llama a `send_image_to_flash` para enviar la imagen capturada al servidor.
 - Muestra en consola la respuesta del servidor o errores de envío.
 - Realiza una espera de 2 segundos entre capturas para estabilidad.

Este módulo permite:

- Capturar imágenes con la cámara OV7670 de forma continua y automática.
- Conectarse a redes WiFi con IP fija, útil para redes de laboratorio.
- Enviar las imágenes a un servidor central Flask de manera organizada, utilizando números de secuencia para trazabilidad.
- Gestionar reintentos de envío para mayor robustez en redes inestables.
- Monitorear el funcionamiento del sistema con mensajes claros en consola.

F. Programa Principal de la Central

`PruebaCentralpoo.py`: Usa la clase `central` para recibir mensajes y reenviar comandos JSON a dos carros específicos. Valida que la red siga activa.

```

1 # archivo: programa_principal.py
2 #PruebaCentralPoo
3 import time
4 from ClaseCentralPoo import CentralWiFi
5
6 # Inicializar clase central con SSID y contrase a
7 central = CentralWiFi(ssid="CentralAP", password="
8     12345678")
9
10 # Bucle principal
11 while True:
12     # Recibir mensaje
13     ip_remitente, puerto_remitente, datos_recibidos
14     = central.recibir_mensaje()
15     mensaje_respuesta = central.get_json_prueba()
16
17     central.enviar_json_a_dispositivo("192.168.4.17"
18         , 5525, mensaje_respuesta)
19     central.enviar_json_a_dispositivo("192.168.4.123
20         ", 5555, mensaje_respuesta)
21
22     # Si hay datos recibidos, hacer algo con ellos
23     if datos_recibidos:
24         print("      Datos recibidos correctamente:
25             ", datos_recibidos)

```

```

# Enviar respuesta o mensaje de prueba

# Chequear estado del AP periodicamente
central.check_wifi()
time.sleep(2)

```

Listing 6: `PruebaCentralpoo.py`

Este archivo ejecuta el **programa principal** que utiliza la clase `CentralWiFi` previamente definida para gestionar la comunicación entre una **central WiFi (Access Point)** y dispositivos clientes que se conectan a ella, en un sistema de control y monitoreo de carros móviles o nodos de red.

A continuación se explica el flujo de trabajo:

- **Importación de módulos:**

- `time` para la temporización del bucle.
- `CentralWiFi` desde `ClaseCentral`, que contiene la clase de gestión de Access Point y UDP.

- **Inicialización del Access Point:** Se crea una instancia de `CentralWiFi` con:

- SSID: "CentralAP"
- Contraseña: "12345678"

Esto configura la Raspberry Pi Pico W como un **Access Point al que se conectarán los carros WiFi para intercambiar datos de control y monitoreo**.

- **Bucle principal de operación:** Dentro de un ciclo infinito se realizan las siguientes tareas:

- 1) **Recepción de mensajes:** Se llama a `central.recibir_mensaje()`, que devuelve:
 - IP del remitente.
 - Puerto del remitente.
 - Datos recibidos en formato JSON si se recibió correctamente.
- 2) **Obtención de mensaje de prueba:** Se genera un mensaje JSON de prueba llamando a `central.get_json_prueba()`, el cual contiene datos de simulación o comandos para enviar a los carros conectados.
- 3) **Envío de mensajes a dispositivos específicos:** Se utiliza:
 - IP destino: 192.168.4.17, puerto 5525
 - IP destino: 192.168.4.123, puerto 5555
 enviando el JSON de prueba generado, simulando un escenario de transmisión de comandos o datos de sincronización a dos carros WiFi de la red.
- 4) **Procesamiento de datos recibidos:** Si se recibieron datos correctamente, se imprimen en consola con un mensaje de confirmación, permitiendo monitorear en tiempo real la información que los carros o nodos envían a la central.
- 5) **Chequeo periódico del estado del AP:** Se llama a `central.check_wifi()` para verificar si el Access Point sigue activo, reactivándolo automáticamente en caso de caída de la red para asegurar robustez del sistema.
- 6) **Temporización:** Se utiliza `time.sleep(2)` para esperar 2 segundos entre cada iteración del bucle,

reduciendo el uso de CPU y permitiendo estabilidad de conexión y procesamiento.

G. Prueba Cliente WiFi

pruebaPooCarrowifi.py: Prueba el módulo cliente en el carro. Recibe comandos y los reenvía.

```
1 #pruebaPooCarrowifi
2 from clase_wifi_module import CarroWiFi
3
4 # Crear una instancia de la clase
5 carro = CarroWiFi(
6     ssid="CentralAP",
7     password="12345678",
8     host="192.168.4.1",
9     port=1234,
10    local_port=5555,
11    my_ip="192.168.4.123",
12    uart_tx=0,
13    uart_rx=1
14 )
15
16 while True:
17     msg = carro.recibir_del_central()
18     if msg:
19         print("        UDP recibido:", msg)
20         carro.send_json(msg)
```

Listing 7: pruebaPooCarrowifi.py

Este archivo implementa el programa principal de un **carro WiFi basado en la Raspberry Pi Pico W** para conectarse al Access Point central, recibir datos por UDP y reenviar información al servidor central o a otros dispositivos según sea necesario.

A continuación se explica su funcionamiento:

- **Importación de la clase:** Se importa `CarroWiFi` desde el módulo `claseWifi`, que contiene toda la lógica de conexión WiFi, manejo de UART, lectura de temperatura y transmisión de datos en JSON.
- **Creación de la instancia del carro:** Se inicializa una instancia de `CarroWiFi` con los siguientes parámetros:
 - `ssid="CentralAP"` y `password="12345678"`: para conectarse al Access Point de la central.
 - `host="192.168.4.1"`: dirección IP de la central.
 - `port=1234`: puerto de la central al que enviará datos.
 - `local_port=5555`: puerto local del carro para recibir mensajes UDP.
 - `my_ip="192.168.4.123"`: IP estática del carro dentro de la red del Access Point.
 - `uart_tx=0, uart_rx=1`: pines para transmisión y recepción UART, conectados a los controladores de movimiento o sensores adicionales en el carro.

Al instanciar el objeto:

- Se configura la UART con los pines indicados.
- Se configura el WiFi en modo cliente con IP estática para conectarse al AP central.

- Se configura un socket UDP para recibir datos en el puerto local configurado.

- **Bucle principal de operación:** Dentro de un ciclo infinito, el carro realiza:

- 1) Llamada a `carro.recibir_del_central()` para verificar si ha llegado un mensaje UDP desde la central.
- 2) Si se recibe un mensaje, este se imprime en consola para monitoreo con el texto " UDP recibido:".
- 3) Luego se llama a `carro.send_json(msg)` para reenviar el mensaje recibido al servidor central o a otros nodos, permitiendo replicar información o confirmaciones de recepción.

H. Programa Principal del Carro AP

PruebaCarroAPPoo.py: Inicializa la clase del carro AP y ejecuta su ciclo principal de escucha y retransmisión.

```
1 # Aqu va tu código Python
2 import machine
3 import time
4
5 def ejemplo():
6     print# archivo: programa_carro_ap.py
7
8 from clase_carro_ap import CarroAP
9
10 carro = CarroAP()
11 carro.ciclo_principal()
```

Listing 8: PruebaCarroAPPoo.py

Este archivo implementa el **programa principal del carro en modo Access Point (AP)** basado en la **Raspberry Pi Pico W**, utilizando la clase `CarroAP` para controlar la comunicación y el procesamiento de datos en la red local.

A continuación se explica su funcionamiento:

- **Importación de la clase:** Se importa la clase `CarroAP` desde el módulo `clase_carro_ap`, la cual gestiona:
 - Configuración del Access Point local del carro.
 - Configuración de un socket UDP para recibir y enviar datos.
 - Lectura de temperatura con el ADC interno.
 - Comunicación con otros dispositivos o módulos mediante UART.
 - Reenvío de mensajes recibidos por UDP o UART a destinos configurados.
- **Instanciación de la clase:** Se crea un objeto `carro` llamando a:

```
carro = CarroAP()
```

utilizando los valores predeterminados de:

- SSID: "carro_ap"
- Contraseña: "12345678"
- Puerto UDP de escucha: 1234
- IP de reenvío: "192.168.4.17"
- Puerto de reenvío: 5555
- IP interna del carro: "192.168.4.123"

permitiendo que el carro cree su propia red WiFi como Access Point, a la que otros dispositivos (por ejemplo, la central o cámaras) puedan conectarse directamente.

- **Ejecución del ciclo principal:** Se llama a:

```
carro.ciclo_principal()
```

el cual ejecuta indefinidamente las siguientes tareas:

- 1) Lectura de temperatura interna y envío por UART a dispositivos conectados para monitoreo.
- 2) Escuchar mensajes por UDP, procesarlos, reenviarlos por UART y enviar confirmaciones al remitente.
- 3) Escuchar mensajes por UART, procesarlos, e interpretar si son para el mismo carro o reenviarlos por UDP a otro destino configurado.
- 4) Dormir brevemente entre iteraciones para mantener estabilidad y evitar sobrecarga del procesador.

VII. RESULTADOS Y COMPARACIÓN

El sistema permitió el envío correcto de estructuras JSON desde la central hacia los carros. La información fue reenviada por UART a los microcontroladores, y se ejecutaron los pasos definidos en la coreografía. La transmisión de imágenes desde los carros hacia el servidor fue exitosa, permitiendo observar secuencias sincronizadas.

Sin embargo, durante las pruebas también se identificaron algunos problemas en la estabilidad de la red WiFi, especialmente en escenarios donde múltiples carros estaban activos simultáneamente. Se observaron pérdidas intermitentes de conectividad y comandos recibidos con retardo o con movimientos mínimos, indicando posibles problemas en la intensidad de señal, saturación del canal o limitaciones del hardware utilizado.

VIII. ALCANCES Y LIMITACIONES

- Este prototipo permite el control básico sincronizado de 8 carros en una red local.
- Las pruebas fueron realizadas en entorno controlado, sin interferencia WiFi significativa.
- No se implementó aún el control en tiempo real ni la respuesta a eventos externos.
- Se identificaron problemas de estabilidad en la conexión WiFi, lo que afectó la consistencia de los comandos recibidos por algunos carros, especialmente aquellos más alejados del punto de acceso.

A. Desafíos Específicos de Pico W

- **Limitaciones de memoria:** Dificultad para manejar buffers grandes de imagen
- **Interferencia RF:** Problemas cuando múltiples Pico W transmiten simultáneamente
- **Estabilidad WiFi:** Desconexiones frecuentes con más de 4 dispositivos
- **Ancho de banda:** Limitado a 2 Mbps en condiciones reales
- **Procesamiento:** Demora en conversión RGB565 a RGB888 en el servidor

B. Soluciones Implementadas

- 1) **Manejo de memoria:** Liberación explícita de buffers después de su uso
- 2) **Comunicación UART:** Búfer circular para evitar pérdida de datos
- 3) **Estabilidad WiFi:** Reconexión automática con monitoreo periódico
- 4) **Transmisión de imágenes:** Compresión en origen y división en paquetes

IX. CONCLUSIONES

- Se logró implementar un sistema de transmisión de comandos desde una central hacia carros usando UDP.
- El uso de JSON permitió una descripción flexible de movimientos y acciones.
- La arquitectura permite escalar el sistema hacia nuevos dispositivos o funcionalidades.
- Se debe mejorar la robustez de la red WiFi para garantizar la recepción oportuna de comandos en todos los carros, especialmente bajo condiciones de carga o distancia.

X. TRABAJO FUTURO

- Implementar compresión JPEG en el Pico W usando port de libjpeg
- Migrar a Protocol Buffers para reducir tamaño de comandos
- Implementar red mesh entre Pico W para mejorar escalabilidad
- Desarrollar controlador WiFi de bajo nivel para mejor estabilidad
- Usar PIO para manejo eficiente de cámara
- Implementar OTA updates para reprogramación remota

REFERENCES

- [1] Raspberry Pi Foundation, "RP2040 Datasheet", 2021.
- [2] MicroPython Documentation, "MicroPython for Raspberry Pi Pico", 2023.
- [3] Infineon, "CYW43439 Single-Chip IEEE 802.11 b/g/n MAC/Baseband/Radio", Datasheet, 2020.
- [4] OmniVision, "OV7670/OV7171 CMOS VGA CameraChip Sensor Datasheet", 2010.
- [5] Damien P. George, "MicroPython: Python for microcontrollers", PyCon Australia, 2014.